

Unit-3

Multimodal Translation and Mapping

MULTIMODAL MACHINE LEARNING

MULTIMODAL AI

Bridging Language, Vision & Sound



Beginning of everything

LANGUAGE MODEL



Language Model

- At its core, a **Language Model (LM)** is a statistical tool that calculates the probability of a sequence of words occurring in a particular order. More simply, it tries to predict the next word in a sequence given the words that have come before it.
- **What it does:**
 - Assigns a probability score to a given sequence of words.
 - Predicts the most likely next word (or words) in a sentence.



Language Model

- "The patient felt much better after taking the prescribed..."

medicine
pills
dosage

*mountain
*television



Why word prediction?

Grammar and Style Correction:

- **User Input:** "The team will meat on Tuesday."
- **LM Correction:** "The team will **meet** on Tuesday."

Predictive Search / Query Expansion:

- **User Input:** "Best rest..."
- **LM Prediction:** "Best **restaurants near me**"



Why word prediction?

Improving Machine Translation:

- **Raw Translation** : "She beautiful house living in."
- **LM Refinement**: "She is living in a beautiful house."

Voice Assistant Understanding:

- **User Spoken Input** : "Set an alarm for ate o'clock."
- **LM Interpretation**: "Set an alarm for **8 o'clock**."
- Many more....



Language Models: A History

Early Beginnings (1940s-1980s):

- Early models were often rule-based, relying on manually crafted grammatical rules. These were rigid and struggled with ambiguity and scale.

The Rise of Statistical Models (1980s-Early 2010s):

- The breakthrough came with the shift towards **statistical methods**. Instead of hand-coding rules, models learned patterns from large text corpora (collections of text).
- **N-gram models** emerged as the dominant approach, leveraging Markov assumptions to simplify probability calculations. These were simpler, more robust, and could be trained on vast amounts of data.
- They were the workhorse for applications like speech recognition throughout this period.



Language Models: A History

The Neural Revolution (Mid-2010s - Present):

- The advent of powerful neural networks (especially Recurrent Neural Networks like LSTMs and later, Transformers) revolutionized LMs.
- Neural LMs can capture much longer-range dependencies and complex linguistic patterns than their statistical predecessors.
- This led to the development of highly sophisticated models like Word2Vec, ELMo, BERT, GPT, and many more, which have dramatically improved performance across nearly all NLP tasks.



Probabilistic Language Models

At their core, all language models are **probabilistic**. They assign a probability to a sequence of words.

Let's consider a sequence of words $W=w_1, w_2, \dots, w_N$.

A probabilistic language model aims to calculate the probability of this entire sequence, $P(W)$ or $P(w_1, w_2, \dots, w_N)$.



Probabilistic Language Models

Goal: compute the probability of a sentence or sequence of words W :

- $P(W) = P(w_1, w_2, w_3, w_4, w_5 \dots w_n)$

Related task: probability of an upcoming word:

- $P(w_5 | w_1, w_2, w_3, w_4)$ or $P(w_n | w_1, w_2 \dots w_{n-1})$

An LM computes either of these:

- $P(W)$ or $P(w_n | w_1, w_2 \dots w_{n-1})$



Probabilistic Language Models

Could we just count and divide?

$P(\text{sky} | \text{The clouds drifted across the})$

- If we tried to calculate this directly by counting:

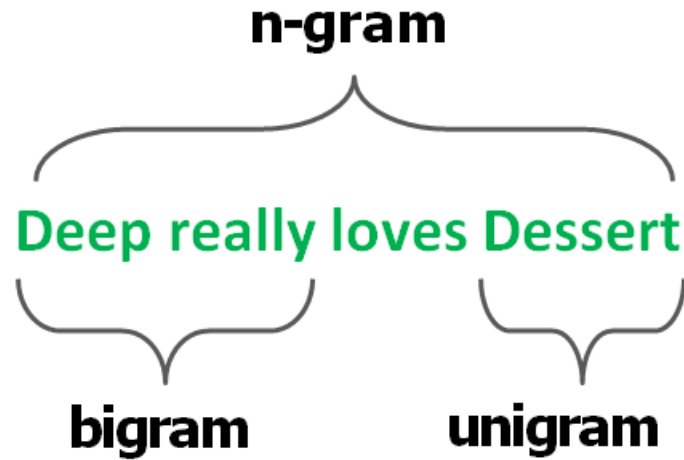
$C(\text{The clouds drifted across the})$

$C(\text{The clouds drifted across the sky})$

- **No! Still too many unique combinations!**
- Even for a five-word sequence like "The clouds drifted across the sky," the likelihood of seeing this *exact sequence* repeatedly in any given training corpus is very low.



N-Grams



Unigrams

- A **unigram** is just a **single word** treated as a unit.

"AI is transforming education"

["AI", "is", "transforming", "education"]

Unigrams

- In early NLP (like spam detection, sentiment analysis), we convert text into:
 - Bag of Words (BoW)
 - TF-IDF vectors
- Each word becomes a feature

Unigrams

- Unigrams **ignore word order**.
 - “not good”
 - “good”
- Both contain "good" → model might think both are positive.
- That's a big problem.

Bigrams

- A **bigram** is a pair of consecutive words.

"AI is transforming education"

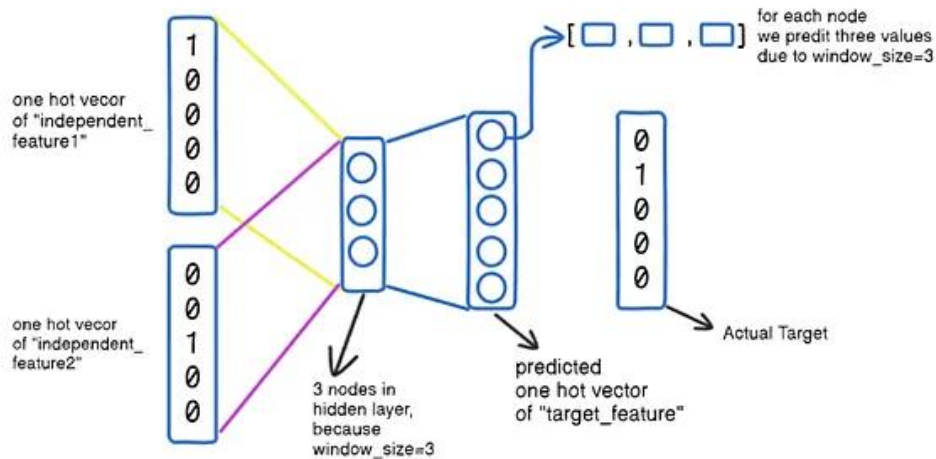
["AI is",
"is transforming",
"transforming education"]

Bigrams

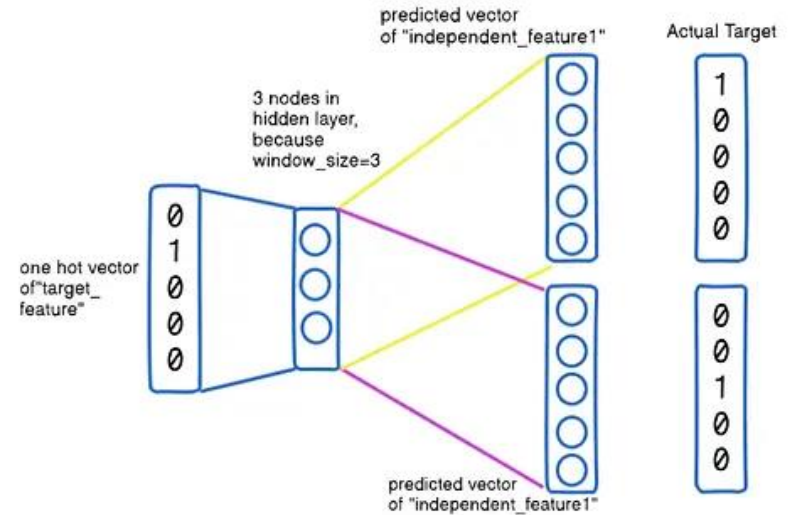
- Only captures 2-word context
- Feature space explodes quickly
- Doesn't generalize well to unseen phrases

Word2vec

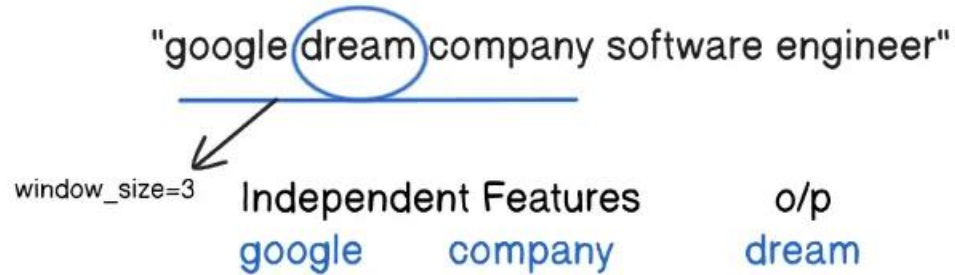
CBOW



Skip-gram



CBOW

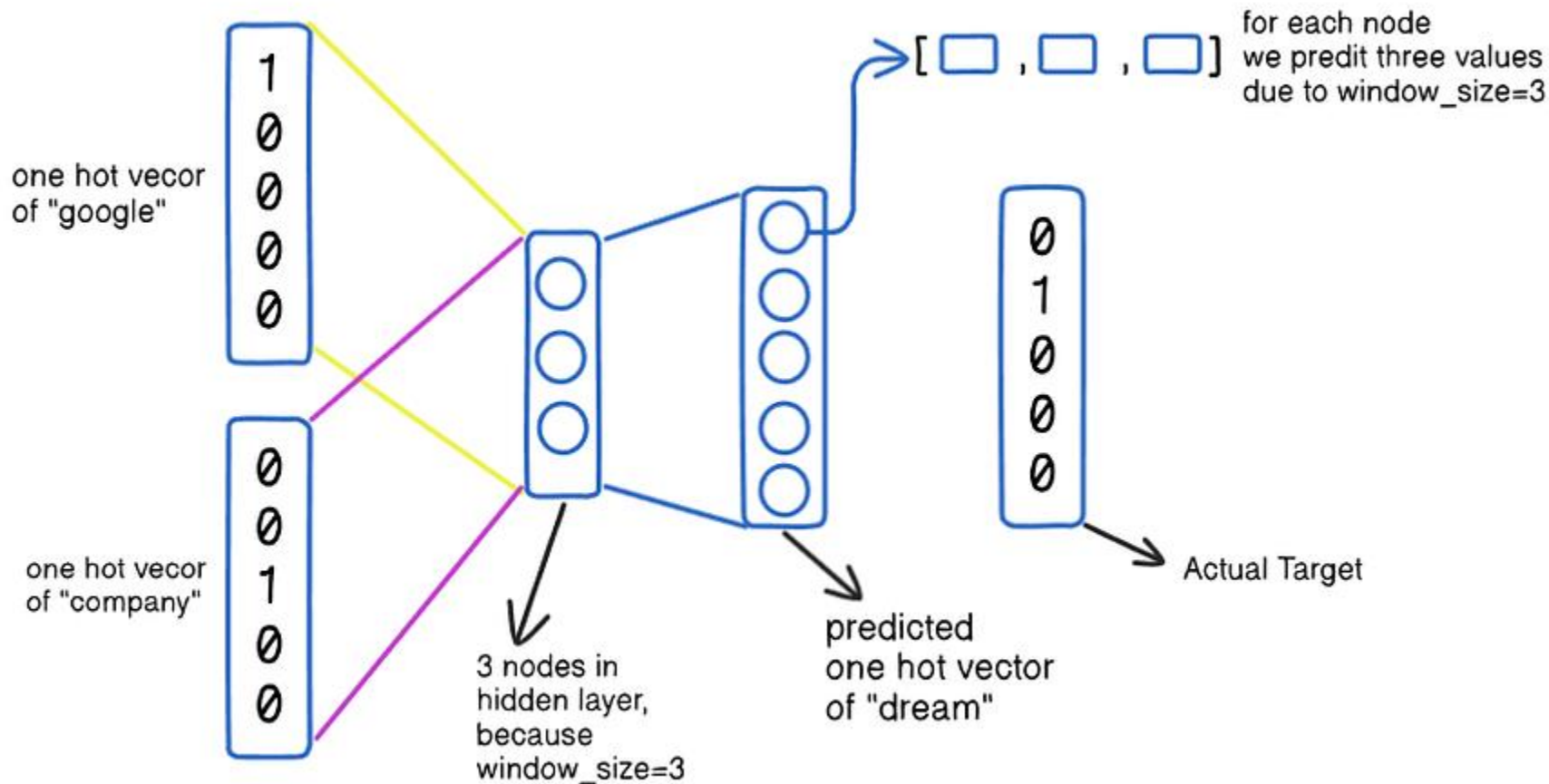


google dream company software engineer

[1 , 0 , 0 , 0 , 0]

[0 , 0 , 1 , 0 , 0]

CBOW



CBOW

Weights after training (W)

w00	w01	w02	w03	w04
w10	w11	w12	w13	w14
w20	w21	w22	w23	w24

one hot vector of words (V)

google	dream	company	software	engineer
1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

Word Vector For google = $W \times V$

w00	w01	w02	w03	w04
w10	w11	w12	w13	w14
w20	w21	w22	w23	w24

 $\times$

1
0
0
0
0

 $=$

w00
w10
w20

Skip-Gram

"google dream company software engineer"

Independent Features

google

company

o/p

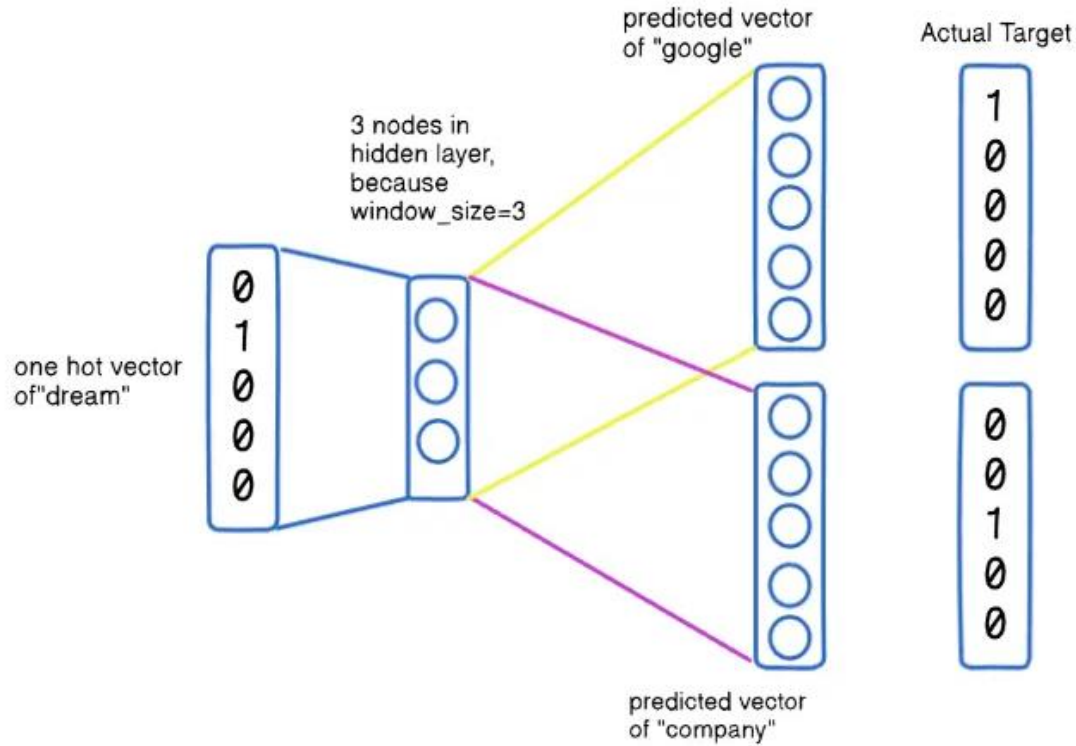
dream

google dream company software engineer

[1 , 0 , 0 , 0 , 0]

[0 , 0 , 1 , 0 , 0]

Skip-Gram



Skip Thought

$x(0)$: Hi, My name is Sanyam

$x(1)$: Today, I went to the zoo.

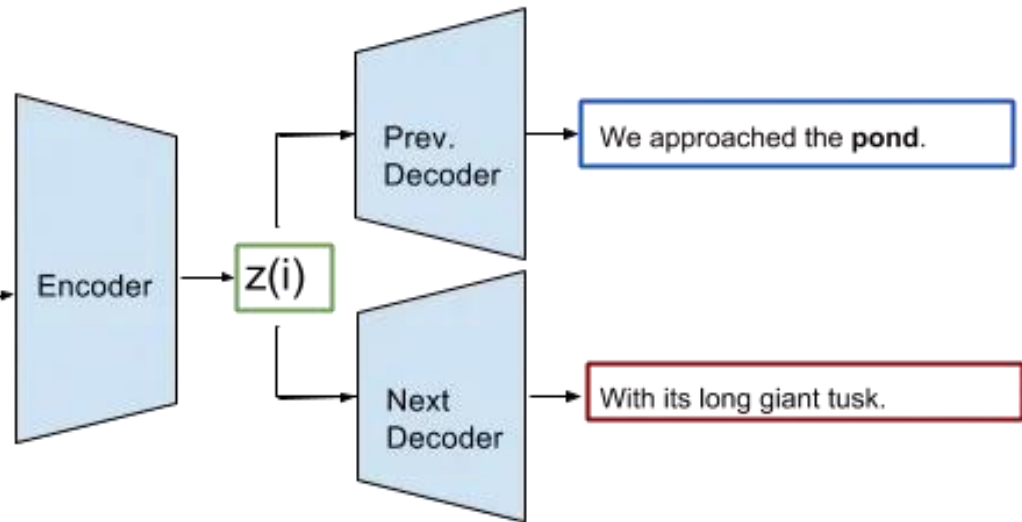
...

$x(i-1)$: We approached the tree.

$x(i)$: The elephant was still.

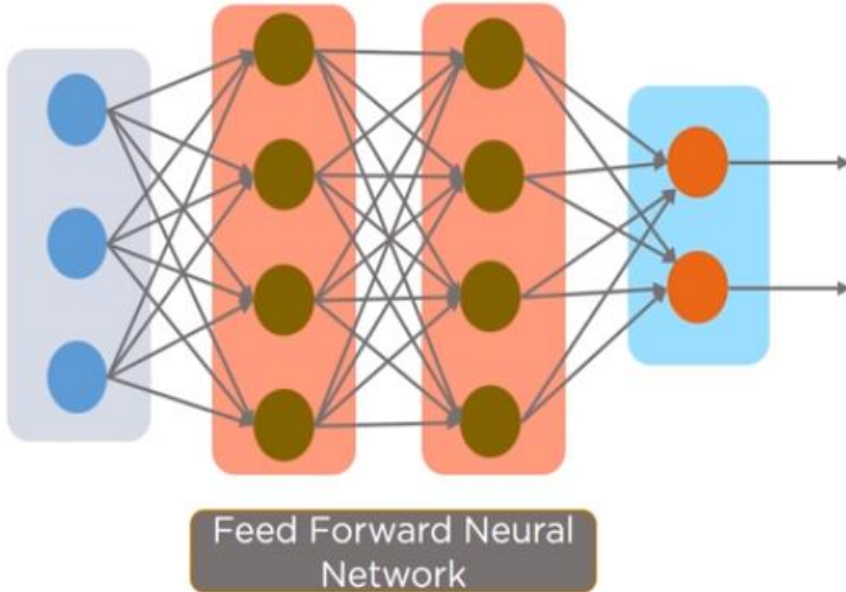
$x(i+1)$: It was taking a nap probably.

...



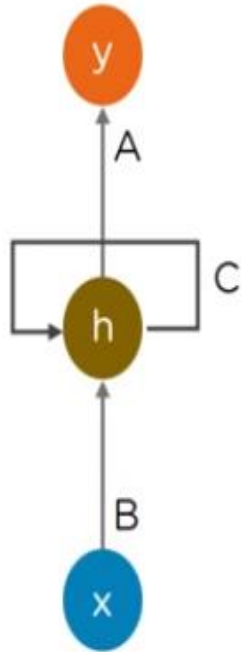
Why RNN ?

Issues in Feed Forward Neural Network



- 01 cannot handle sequential data
- 02 considers only the current input
- 03 cannot memorize previous inputs

Why RNN?



Recurrent Neural
Network



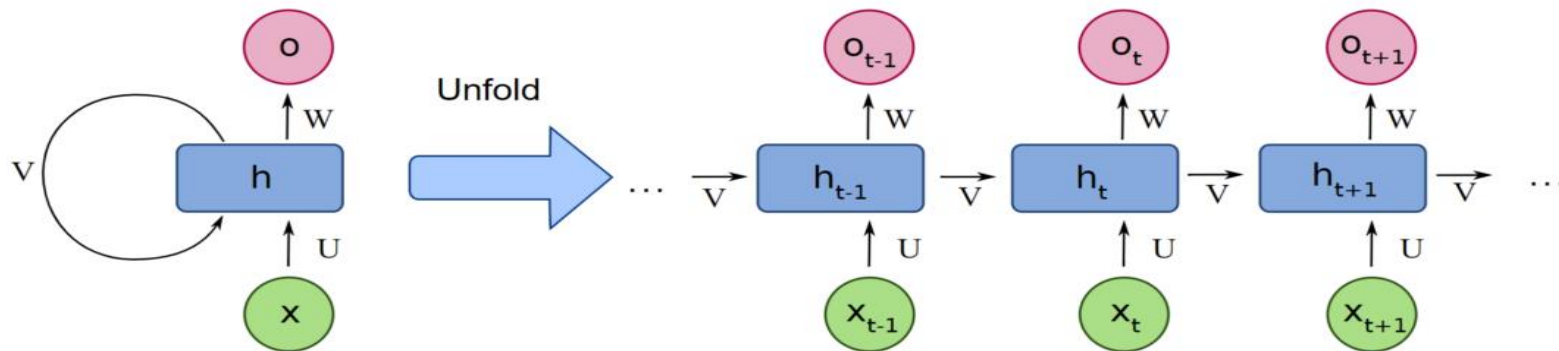
- 01 can handle sequential data
- 02 considers the current input and also the previously received inputs
- 03 can memorize previous inputs due to its internal memory

Types of Sequence Tasks

Task Type	Example
One-to-One	Sentiment classification
One-to-Many	Image → Caption
Many-to-One	Text → Sentiment
Many-to-Many	Translation
Many-to-Many (aligned)	POS tagging

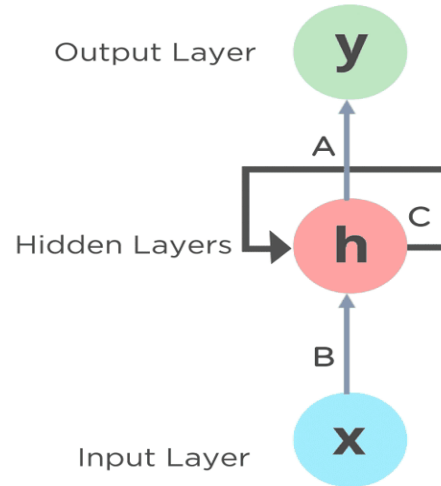
Recurrent Neural Networks (RNN)

- A Deep Learning approach for modelling sequential data.
- They work especially well for jobs requiring sequences, such as time series data, voice, natural language, and other activities.



Recurrent Neural Networks (RNN)

- RNN works on the principle of saving the output of a particular layer and feeding this back to the input in order to predict the output of the layer.
- The nodes in different layers of the neural network are compressed to form a single layer of recurrent neural networks. A , B , and C are the parameters of the network.



Rotate
anticlockwise and
compress the
layers.

A , B and C are the parameters

Recurrent Neural Network

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

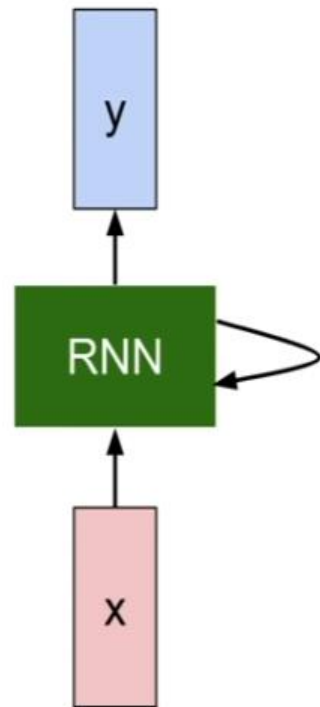
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

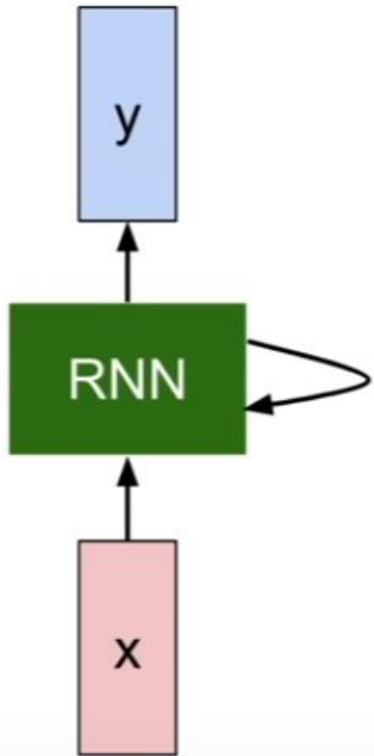
some function with parameters W

old state

input vector at some time step



Recurrent Neural Network



$$h_t = f_W(h_{t-1}, x_t)$$



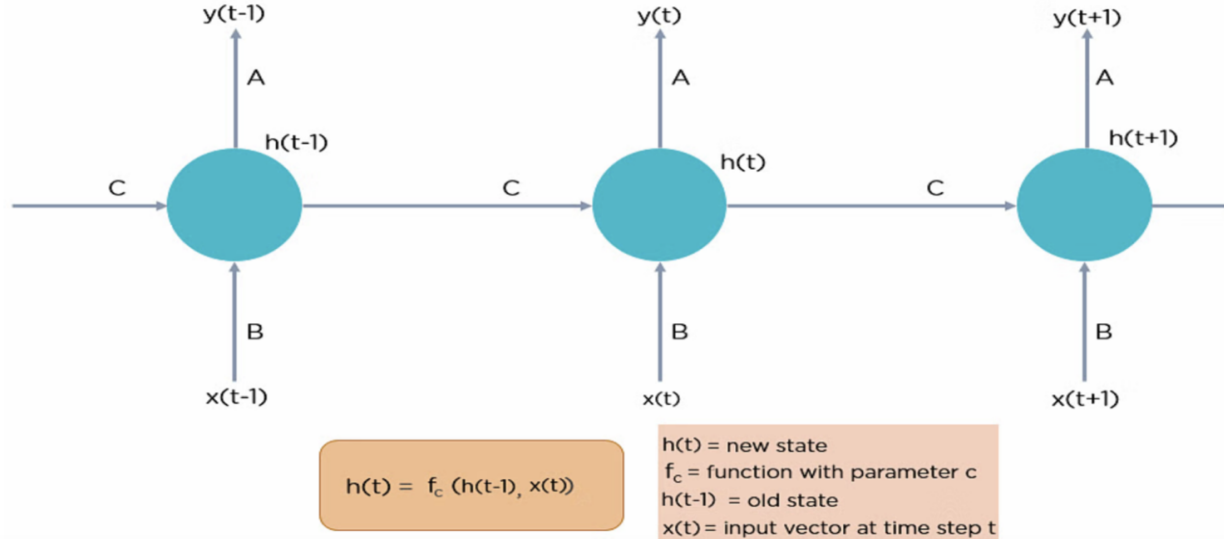
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

Recurrent Neural Networks (RNN)

Architecture

- RNNs are a type of neural network that has hidden states and allows past outputs to be used as inputs.
- They usually go like this:

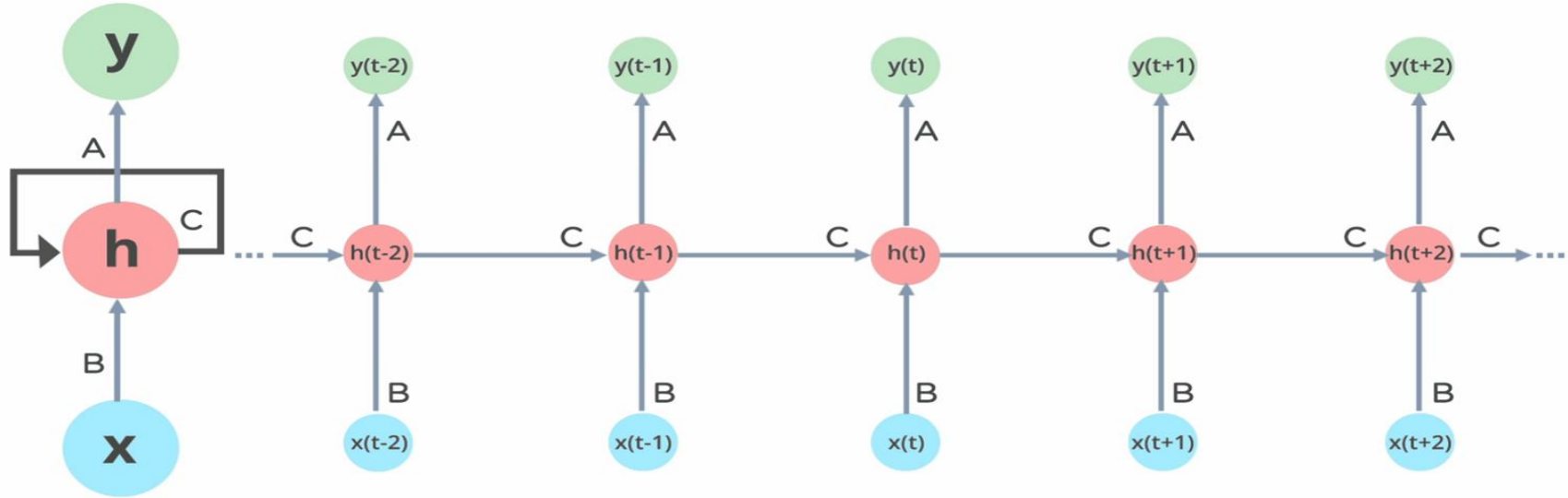


“x” is the input layer
“h” is the hidden layer
“y” is the output layer
A, B, and C are the network parameters used to improve the output of the model

The output at any given time is fetched back to the network to improve on the output.

Recurrent Neural Networks (RNN)

Working



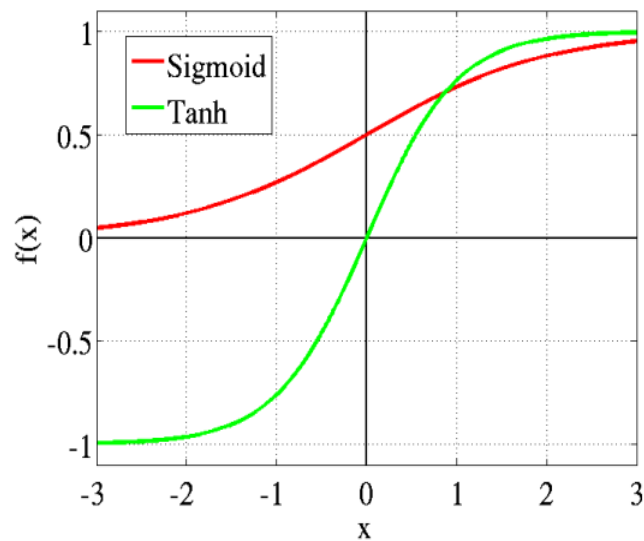
- The RNN will standardize the different activation functions and weights and biases so that each hidden layer has the same parameters.
- Then, instead of creating multiple hidden layers, it will create one and loop over it as many times as required.

Gradient Problem

- Exploding Gradients
- Vanishing Gradients

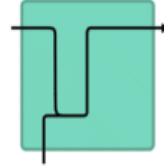
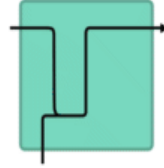
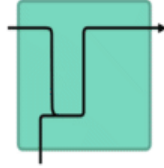
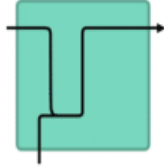


Tanh vs Sigmoid



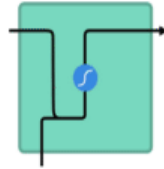
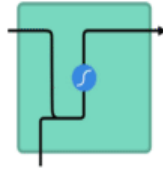
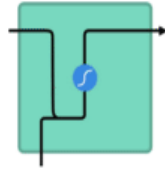
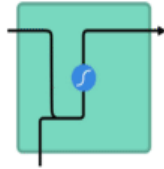
Without Tanh

5
0.01
-0.5



With Tanh

5
0.01
-0.5



Customers Review 2,491

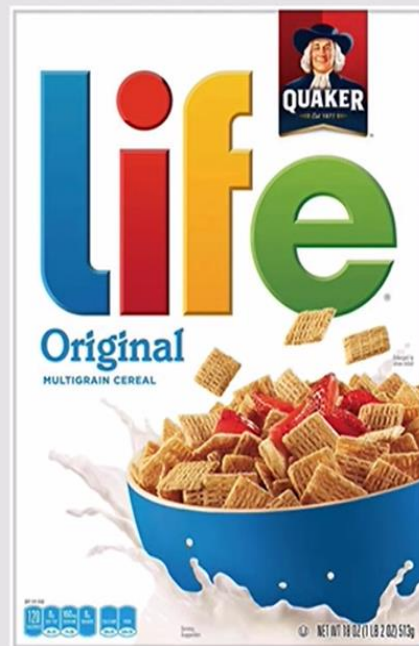


Thanos

September 2018

Verified Purchase

Amazing! This box of cereal gave me a perfectly balanced breakfast, as all things should be. I only ate half of it but will definitely be buying again!



A Box of Cereal

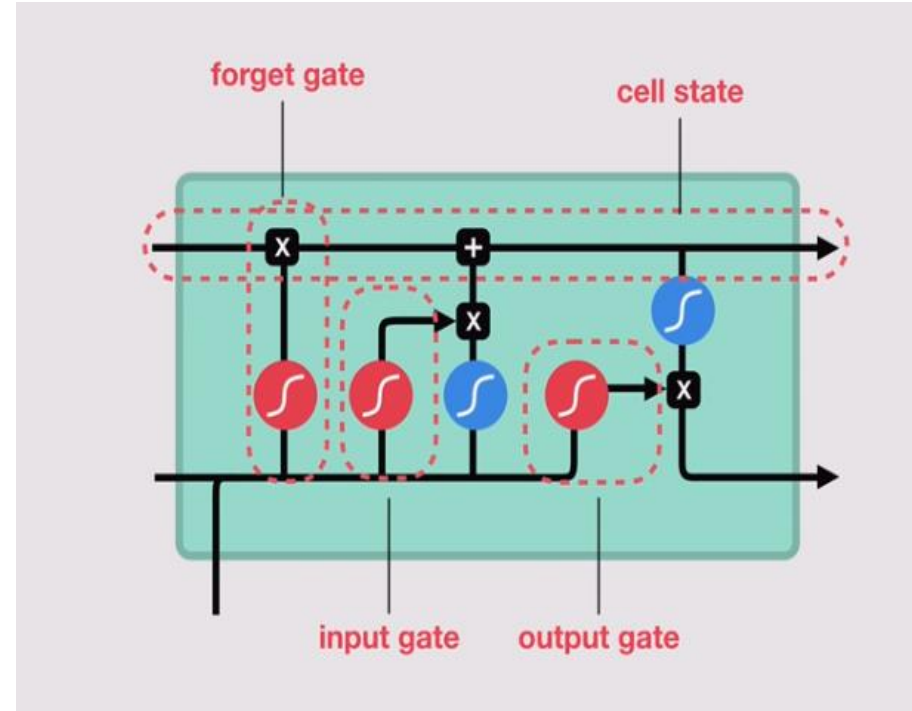
\$3.99

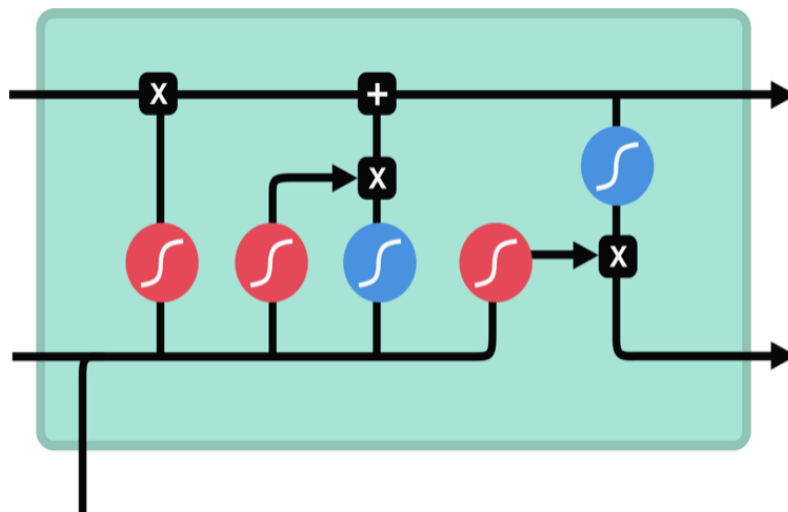
LSTM

Forget Gate: conditionally decides what information to discard from the unit.

Input Gate: conditionally decides which values from the input to update the memory state.

Output Gate: conditionally decides what to output based on input and the memory of the unit.





sigmoid



tanh



pointwise
multiplication

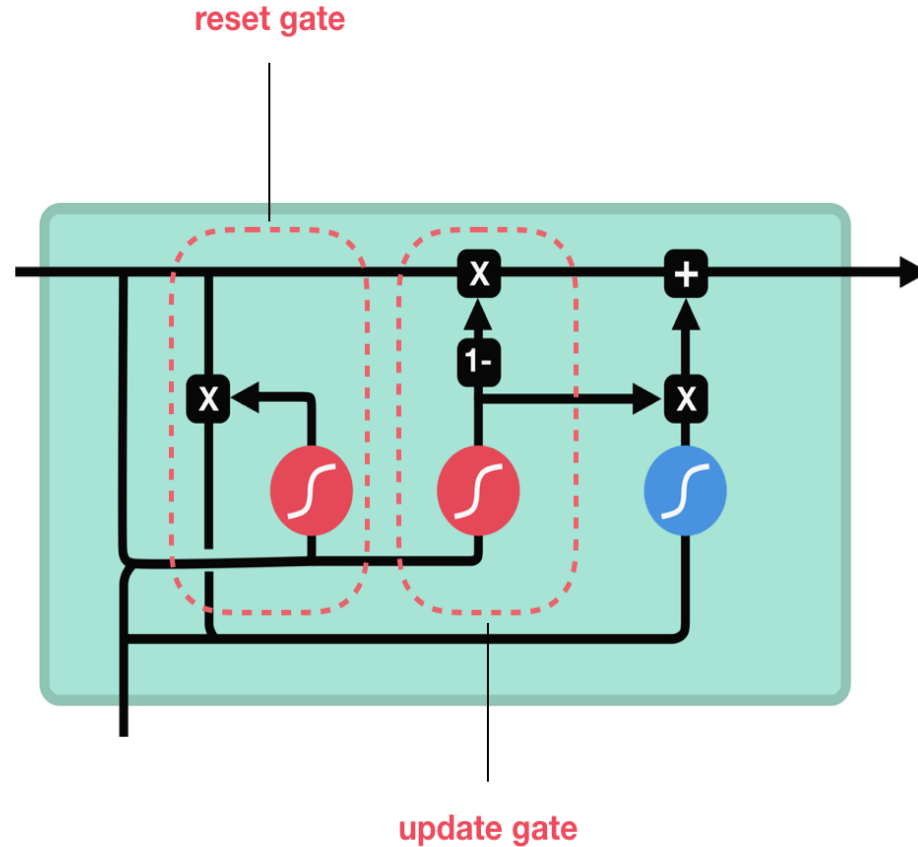


pointwise
addition



vector
concatenation

GRU – Gated Recurrent Unit



End of Slide

THANK YOU

